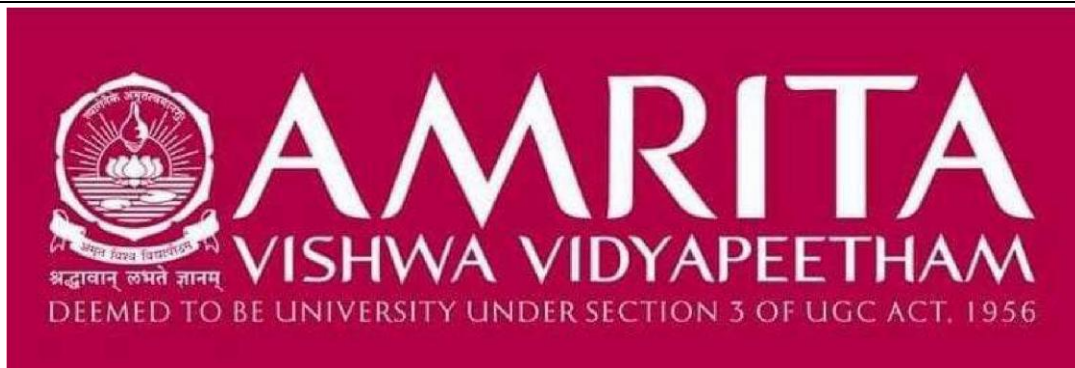




DATA STRUCTURES AND ALGORITHMS 23CSE203 LAB MANUAL

SUBMITTED BY	
NAME	Saket S
ROLL NO.	AV.SC.U4CSE25048
YEEAR/SEM/SECTION	2 nd YEAR /SEM-3/CSE-A
CREDITS	05
GIT REPO LINK	

SUBMITTED TO	DR.RAJ KUMAR BATCHU
DESIGNATION	ASST.PROFFESSOR(S 1 .Grade)
DEPARTMENT	School of Computing

INDEX

	TITLE	PAGE NO	REMARKS
1	Lab1:- 1a) rocket launch systems display the countdown of before launch instead of using a loop implement the countdown using the recursion write a recursion Python program to display the countdown from n to 1 followed by launch. 1b) Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive Python program to calculate P^n (power), where P = Principal growth factor, n = number of years. 1c) An HR department stores employee IDs in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list. 1d) In a logistics company, the number of possible ways to arrange n different parcels is calculated using the factorial formula. Develop a recursive program to calculate the factorial of a given number. 1e) Write a recursive Python program to print the first n terms of a Fibonacci series.	3-12	
2			
3			
4			
5			

LAB-1

1A)AIM:- rocket launch systems display the countdown of before launch instead of using a loop implement the countdown using the recursion write a recursion Python program to display the countdown from n to 1 followed by launch.

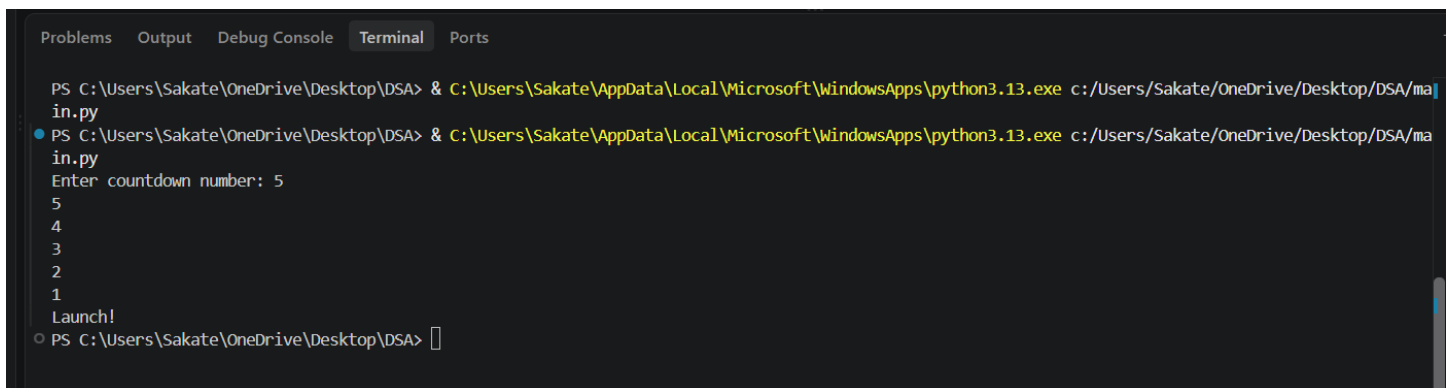
PROGRAM:-

```
def countdown(n):
    if n == 0:
        print("Launch!")
    else:
        print(n)
        countdown(n - 1)

# Input
n = int(input("Enter countdown number: "))

# Function call
countdown(n)
```

OUTPUT:-



```
Problems Output Debug Console Terminal Ports
PS C:\Users\Sakate\OneDrive\Desktop\DSA> & C:\Users\Sakate\AppData\Local\Microsoft\WindowsApps\python3.13.exe c:/Users/Sakate/OneDrive/Desktop/DSA/main.py
PS C:\Users\Sakate\OneDrive\Desktop\DSA> & C:\Users\Sakate\AppData\Local\Microsoft\WindowsApps\python3.13.exe c:/Users/Sakate/OneDrive/Desktop/DSA/main.py
Enter countdown number: 5
5
4
3
2
1
Launch!
PS C:\Users\Sakate\OneDrive\Desktop\DSA> 
```

EXPLANATION:-

- def defines the countdown() function.
- input() accepts the countdown value from the user.
- int() converts the input into an integer.
- countdown() starts the recursive countdown process.
- if checks the base case (n == 0).
- else executes when the condition is False.
- print(n) displays the current countdown number.
- countdown(n - 1) recursively calls the function with the value decreased by 1.
- print("Launch!") displays the launch message when the countdown reaches 0.
- The program demonstrates the use of functions, recursion, conditional statements, input, and output to perform the countdown.

ERRORS:-

SNO	Errors Observed	Rectification
1	Invalid (non-numeric) input entered by the user.	Enter a valid integer as input.
2	Missing base case (if n == 0) causing infinite recursion.	Include the base case to stop recursion .l
3	incorrect indentation leading to an Indentation Error	Maintain proper Python indentation.
4	Missing colon (:) after def, if, or else.	Add the required colon (:) after statements.
5	Calling countdown(n) instead of countdown(n - 1), causing infinite recursion.	Decrease the value by 1 in each recursive call using countdown(n - 1).

TIME COMPLEXITY:- $O(n)$

SPACE COMPLEXITY:- $O(n)$

REASON:- Function is called once for each value from n to 0.

•

1B)AIM:- Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive Python program to calculate P^n (power), where P = Principal growth factor, n = number of years.

PROGRAM:-

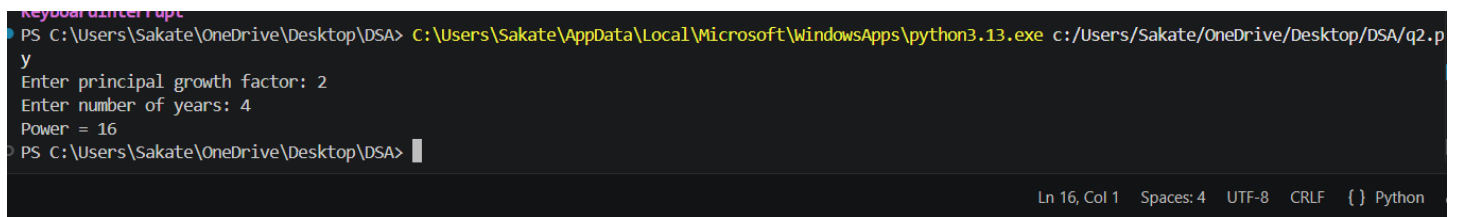
```
def power(p, n):
    if n == 0:
        return 1
    else:
        return p * power(p, n - 1)

# Input
p = int(input("Enter principal growth factor: "))
n = int(input("Enter number of years: "))

# Function call
result = power(p, n)

# Output
print("Power =", result)
```

OUTPUT:-



```
keyba...interrupt
PS C:\Users\Sakate\OneDrive\Desktop\DSA> C:\Users\Sakate\AppData\Local\Microsoft\WindowsApps\python3.13.exe c:/Users/Sakate/OneDrive/Desktop/DSA/q2.py
Enter principal growth factor: 2
Enter number of years: 4
Power = 16
PS C:\Users\Sakate\OneDrive\Desktop\DSA>
```

Ln 16, Col 1 Spaces: 4 UTF-8 CRLF { } Python

EXPLANATION:-

- `power()` is the recursive function that computes P raised to n .
- The base case is $n == 0$, which returns 1 (any number to the power 0 is 1).
- The recursive case multiplies p with `power(p, n - 1)`, reducing n by 1 on each call.
- `float()` allows the principal growth factor to be a decimal value.
- `input()` collects the values of P and n from the user.
- Each recursive call is stacked until the base case is reached, then the results are multiplied back up the call stack.
- `print()` displays the final computed value of P^n .

ERRORS:-

SNO	Errors Observed	Rectification
1	Missing base case ($n == 0$) causing infinite recursion.	Include the base case to stop recursion.
2	Entering a negative value for n causes the recursion to never reach 0.	Restrict input to non-negative integers or handle negative n separately.
3	Using <code>int()</code> for P truncates decimal growth factors.	Use <code>float()</code> to accept decimal values for P .
4	Incorrect indentation leading to an <code>IndentationError</code> .	Maintain proper Python indentation.
5	Calling <code>power(p, n)</code> instead of <code>power(p, n - 1)</code> , causing infinite recursion.	Decrease n by 1 in each recursive call using <code>power(p, n - 1)</code> .

TIME COMPLEXITY:- $O(n)$

SPACE COMPLEXITY:- $O(n)$

REASON:- Recursive call is made n times to compute P^n .

•

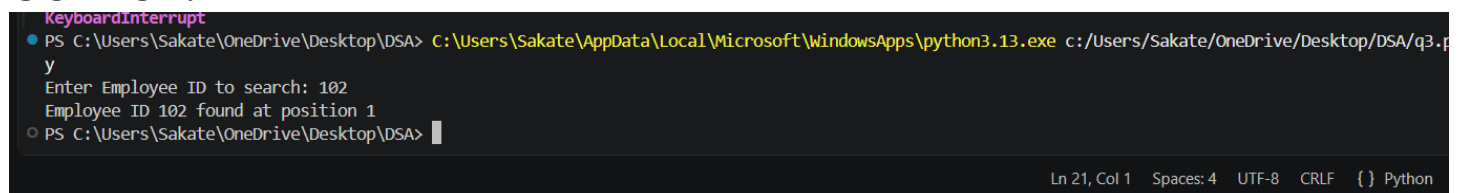
1C)AIM:- An HR department stores employee IDs in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list.

PROGRAM:-

```
def search_employee(emp_list, emp_id, index=0):  
    if index == len(emp_list):  
        return -1  
    elif emp_list[index] == emp_id:  
        return index  
    else:  
        return search_employee(emp_list, emp_id, index + 1)
```

```
# Input  
employee_ids = [101, 102, 103, 104, 105]  
search_id = int(input("Enter Employee ID to search: "))  
  
# Function call  
position = search_employee(employee_ids, search_id)  
  
if position != -1:  
    print("Employee ID", search_id, "found at position", position)  
else:  
    print("Employee ID", search_id, "not found")
```

OUTPUT:-



```
KeyboardInterrupt  
PS C:\Users\Sakate\OneDrive\Desktop\DSA> C:\Users\Sakate\AppData\Local\Microsoft\WindowsApps\python3.13.exe c:/Users/Sakate/OneDrive/Desktop/DSA/q3.py  
Enter Employee ID to search: 102  
Employee ID 102 found at position 1  
PS C:\Users\Sakate\OneDrive\Desktop\DSA> |  
Ln 21, Col 1  Spaces: 4  UTF-8  CRLF  { } Python
```

EXPLANATION:-

- `search_employee()` is the recursive function that searches the list one index at a time.
- The base case `index == len(emp_list)` means every element was checked and the ID was not found, so it returns -1.
- The second condition checks if the current element matches the employee ID being searched.
- If not matched, the function calls itself with `index + 1`, moving to the next employee in the list.
- `employee_ids` is the list of employee IDs stored by the HR department.
- `input()` and `int()` collect the employee ID to search for from the user.
- The if-else block reports whether the employee ID was found and at which position.

ERRORS:-

SNO	Errors Observed	Rectification
1	Missing base case (<code>index == len(emp_list)</code>) causing infinite recursion.	Include the base case to stop recursion when the list is exhausted.
2	Invalid (non-numeric) input entered by the user.	Enter a valid integer as input.
3	Calling <code>search_employee(emp_list, emp_id, index)</code> without incrementing index, causing infinite recursion.	Increase index by 1 in each recursive call using <code>index + 1</code> .
4	Incorrect indentation leading to an <code>IndentationError</code> .	Maintain proper Python indentation.
5	Searching an empty list without handling it, giving a confusing result.	Add a check for an empty list before searching.

TIME COMPLEXITY:- $O(n)$

SPACE COMPLEXITY:- $O(n)$

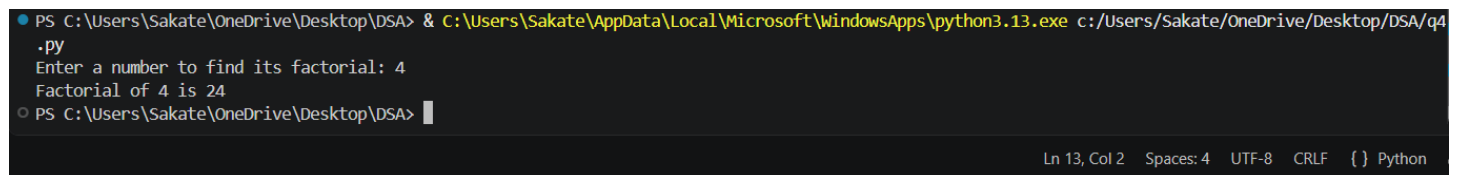
REASON:- In the worst case, every element in the list is checked recursively.

1D)AIM:- In a logistics company, the number of possible ways to arrange n different parcels is calculated using the factorial formula. Develop a recursive program to calculate the factorial of a given number.

PROGRAM:-

```
def factorial(n):  
    if n == 0 or n == 1:  
        return 1  
    else:  
        return n * factorial(n - 1)  
  
# Input  
n = int(input("Enter a number to find its factorial: "))  
  
# Function call  
print("Factorial of", n, "is", factorial(n))
```

OUTPUT:-



```
PS C:\Users\Sakate\OneDrive\Desktop\DSA> & C:\Users\Sakate\AppData\Local\Microsoft\WindowsApps\python3.13.exe c:/Users/Sakate/OneDrive/Desktop/DSA/q4  
.py  
Enter a number to find its factorial: 4  
Factorial of 4 is 24  
PS C:\Users\Sakate\OneDrive\Desktop\DSA> |
```

Ln 13, Col 2 Spaces: 4 UTF-8 CRLF { } Python

EXPLANATION:-

- factorial() is the recursive function that computes n! (n factorial).
- The base case is $n == 0$ or $n == 1$, both of which return 1.
- The recursive case multiplies n with factorial(n - 1), reducing n by 1 on each call.
- input() and int() collect the number whose factorial is to be found.
- Each recursive call is stacked until the base case is reached, then the results are multiplied back up the call stack.
- print() displays the number of possible arrangements (the factorial value).

ERRORS:-

SNO	Errors Observed	Rectification
1	Missing base case ($n == 0$ or $n == 1$) causing infinite recursion.	Include the base case to stop recursion.
2	Entering a negative number, for which factorial is undefined.	Restrict input to non-negative integers.
3	Invalid (non-numeric) input entered by the user.	Enter a valid integer as input.
4	Incorrect indentation leading to an IndentationError.	Maintain proper Python indentation.
5	Calling factorial(n) instead of factorial(n - 1), causing infinite recursion.	Decrease n by 1 in each recursive call using factorial(n - 1).

TIME COMPLEXITY:- $O(n)$

SPACE COMPLEXITY:- $O(n)$

REASON:- Function calls itself n times until reaching the base case.

1E)AIM:- Write a recursive Python program to print the first n terms of a Fibonacci series.

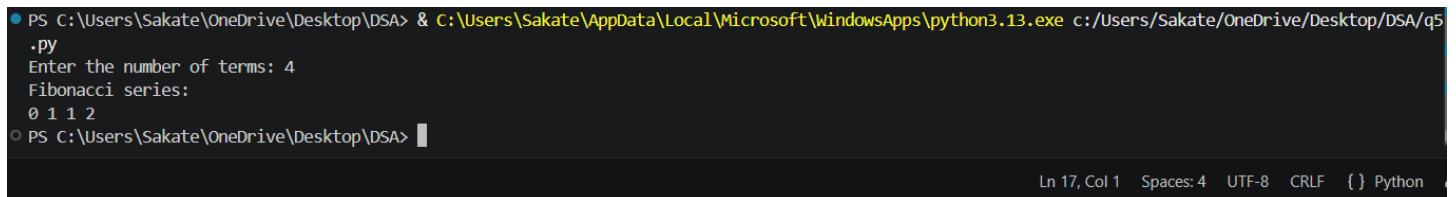
PROGRAM:-

```
def fibonacci(n):
    if n == 0:
        return 0
    elif n == 1:
        return 1
    else:
        return fibonacci(n - 1) + fibonacci(n - 2)

# Input
n = int(input("Enter the number of terms: "))

# Function call
print("Fibonacci series:")
for i in range(n):
    print(fibonacci(i), end=" ")
```

OUTPUT:-



```
PS C:\Users\Sakate\OneDrive\Desktop\DSA> & C:\Users\Sakate\AppData\Local\Microsoft\WindowsApps\python3.13.exe c:/Users/Sakate/OneDrive/Desktop/DSA/q5
.py
Enter the number of terms: 4
Fibonacci series:
0 1 1 2
PS C:\Users\Sakate\OneDrive\Desktop\DSA>
```

EXPLANATION:-

- fibonacci() is the recursive function that returns the nth Fibonacci term.
- The base cases are $n == 0$ (returns 0) and $n == 1$ (returns 1).
- The recursive case adds $\text{fibonacci}(n - 1)$ and $\text{fibonacci}(n - 2)$, following the Fibonacci rule.
- input() and int() collect the number of terms to be printed.
- The for loop calls fibonacci(i) for each index from 0 to $n - 1$.
- print(..., end=" ") displays each term on the same line, separated by a space.

ERRORS:-

SNO	Errors Observed	Rectification
1	Missing base cases ($n == 0$ and $n == 1$) causing infinite recursion.	Include both base cases to stop recursion.
2	Entering a negative number of terms.	Restrict input to non-negative integers.
3	Invalid (non-numeric) input entered by the user.	Enter a valid integer as input.
4	Incorrect indentation leading to an IndentationError.	Maintain proper Python indentation.
5	Using $\text{fibonacci}(n - 1) - \text{fibonacci}(n - 2)$ instead of addition, giving wrong terms.	Use addition, $\text{fibonacci}(n - 1) + \text{fibonacci}(n - 2)$, as per the Fibonacci rule.

TIME COMPLEXITY:- $O(2^n)$

SPACE COMPLEXITY:- $O(n)$

REASON:- Each call creates two recursive calls, resulting in exponential time, while the recursion depth is n .

LAB-2

2A)AIM:- Write a Python program to perform linear search for the given input.

PROGRAM:-

```
def linear_search(arr, target):
    for index in range(len(arr)):
        if arr[index] == target:
            return index
    return -1

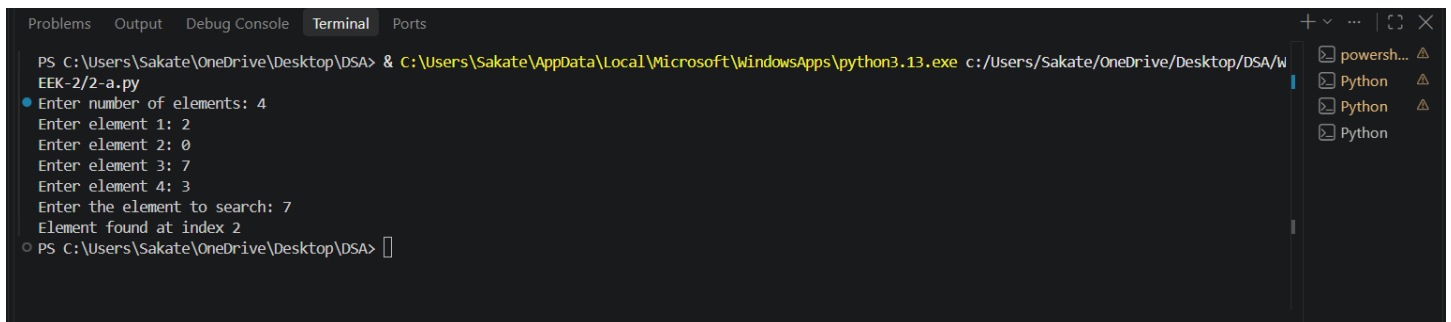
# Input n = int(input("Enter number of
elements: ")) arr = [] for i in range(n):
    arr.append(int(input(f'Enter element {i+1}: ")))

target = int(input("Enter the element to search: "))

# Function call result =
linear_search(arr, target)

# Output
if result != -1:
    print("Element found at index",
result) else:    print("Element not found
in the list")
```

OUTPUT:-



The screenshot shows a terminal window with the following text:

```
PS C:\Users\Sakate\OneDrive\Desktop\DSA> & C:\Users\Sakate\AppData\Local\Microsoft\WindowsApps\python3.13.exe c:/Users/Sakate/OneDrive/Desktop/DSA/w
EEK-2/2-a.py
Enter number of elements: 4
Enter element 1: 2
Enter element 2: 0
Enter element 3: 7
Enter element 4: 3
Enter the element to search: 7
Element found at index 2
PS C:\Users\Sakate\OneDrive\Desktop\DSA>
```

The terminal window has tabs for Problems, Output, Debug Console, Terminal, and Ports. The Terminal tab is active. On the right side, there is a sidebar with icons for powershell, Python, and another Python instance.

EXPLANATION:-

- `linear_search()` checks each element of the list one by one from the beginning.
- The for loop iterates over every index of the array using `range(len(arr))`.
- If `arr[index]` equals the target, the matching index is returned immediately.
- If the loop finishes without a match, -1 is returned to indicate the element is not present.
- `input()` and `int()` collect the number of elements, the elements themselves, and the target value.
- The if-else block reports whether the target was found and at which index, or that it was not found.

ERRORS:-

SNO	Errors Observed	Rectification
1	Invalid (non-numeric) input entered by the user.	Enter a valid integer as input.
2	Entering more or fewer elements than the specified count n.	Ensure exactly n elements are entered, one per prompt.
3	Searching an empty list ($n = 0$) without handling it.	Add a check for an empty list before searching.
4	Incorrect indentation leading to an <code>IndentationError</code> .	Maintain proper Python indentation.
5	Using <code>==</code> to compare elements of different types (e.g. <code>int</code> vs <code>str</code>), causing a false "not found".	Ensure the target and list elements are converted to the same type (<code>int</code>) before comparing.

TIME COMPLEXITY:- $O(n)$

SPACE COMPLEXITY:- $O(1)$

REASON:- In the worst case, every element in the list is checked once; no extra space is used besides a few variables

•

2B)AIM:- Write a Python program to perform binary search of the given input, verifying by taking a sorted array and an unsorted array as input.

PROGRAM:-

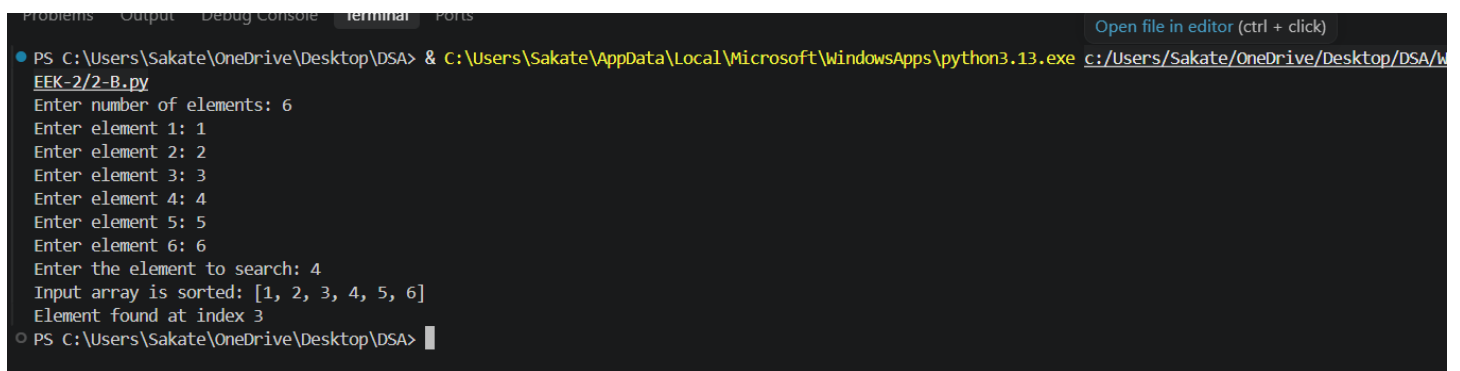
```
def binary_search(arr, target):
    low = 0    high =
len(arr) - 1    while
low <= high:
    mid = (low + high) // 2
    if arr[mid] == target:
        return mid
    elif arr[mid] < target:
        low = mid +
1    else:
        high = mid - 1
    return -1

# Input n = int(input("Enter number of
elements: ")) arr = [] for i in range(n):
    arr.append(int(input(f"Enter element {i+1}: ")))

target = int(input("Enter the element to search: "))

# Verify whether the input array is sorted or unsorted
if arr == sorted(arr):
    print("Input array is sorted:", arr)
    result = binary_search(arr, target)
    if result != -1:
        print("Element found at index",
result)    else:    print("Element not
found in the list") else:
    print("Input array is NOT sorted:", arr)    print("Binary search requires a
sorted array.")    print("Result on unsorted array cannot be trusted. Sorting
the array first...")    sorted_arr = sorted(arr)    print("Sorted array:",
sorted_arr)    result = binary_search(sorted_arr, target)    if result != -1:
        print("Element found at index", result, "in the sorted
array")    else:    print("Element not found in the list")
```

OUTPUT:-



```
PS C:\Users\Sakate\OneDrive\Desktop\DSA> & C:\Users\Sakate\AppData\Local\Microsoft\WindowsApps\python3.13.exe c:/Users/Sakate/OneDrive/Desktop/DSA/EEK-2/2-B.py
Enter number of elements: 6
Enter element 1: 1
Enter element 2: 2
Enter element 3: 3
Enter element 4: 4
Enter element 5: 5
Enter element 6: 6
Enter the element to search: 4
Input array is sorted: [1, 2, 3, 4, 5, 6]
Element found at index 3
PS C:\Users\Sakate\OneDrive\Desktop\DSA>
```

EXPLANATION:-

- `binary_search()` repeatedly narrows the search range between low and high using the midpoint mid.
- If `arr[mid]` equals the target, the index mid is returned immediately.
- If `arr[mid]` is less than the target, the search continues in the right half by moving low to mid + 1.
- Otherwise, the search continues in the left half by moving high to mid - 1.
- `arr == sorted(arr)` checks whether the entered array is already sorted, to verify binary search's precondition.
- If the array is sorted, binary search runs directly on it and reports the result.
- If the array is unsorted, the program shows that binary search cannot be trusted on it, sorts the array with `sorted()`, and re-runs binary search on the sorted copy.

ERRORS:-

SNO	Errors Observed	Rectification
1	Running binary search directly on an unsorted array gives an unreliable or wrong result.	Always verify with <code>arr == sorted(arr)</code> and sort the array before searching.
2	Using <code>mid = (low + high) / 2</code> gives a float, causing a <code>TypeError</code> when indexing the list.	Use integer floor division (<code>//</code>) instead of <code>/</code> to compute mid.
3	Using <code>low < high</code> instead of <code>low <= high</code> , missing the case where low equals high.	Use <code>low <= high</code> so a single remaining element is still checked.
4	Not updating low or high inside the loop, causing an infinite loop.	Always set <code>low = mid + 1</code> or <code>high = mid - 1</code> inside the loop body.
5	Invalid (non-numeric) input entered by the user.	Enter a valid integer as input.

TIME COMPLEXITY:- $O(\log n)$

SPACE COMPLEXITY:- $O(1)$

REASON:- The search range is halved on every iteration, so the number of comparisons grows logarithmically with n ; only a few variables (low, high, mid) are used regardless of input size.

LAB-3

3A)AIM:- Write a Python program to bubble sort the elements, taking input from the user.

PROGRAM:-

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n - 1):
        for j in range(n - 1 - i):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr

# Input
n = int(input("Enter number of elements: "))
arr = []
for i in range(n):
    arr.append(int(input(f"Enter element {i+1}: ")))

# Function call
sorted_arr = bubble_sort(arr)

# Output
print("Sorted array:", sorted_arr)
```

OUTPUT:-

```
PS C:\Users\Sakate\OneDrive\Desktop\DSA> & C:\Users\Sakate\AppData\Local\Microsoft\WindowsApps\python3.13.exe c:/Users/Sakate/OneDrive/Desktop/DSA/WORK-3/3-A.py
Enter number of elements: 4
Enter element 1: 2
Enter element 2: 5
Enter element 3: 1
Enter element 4: 6
Sorted array: [1, 2, 5, 6]
PS C:\Users\Sakate\OneDrive\Desktop\DSA>
```

EXPLANATION:-

- `bubble_sort()` repeatedly steps through the list, comparing adjacent elements.
- The outer loop runs $n - 1$ times, controlling the number of passes needed.
- The inner loop compares `arr[j]` and `arr[j + 1]`, swapping them if they are out of order.
- After each pass, the largest remaining unsorted element "bubbles up" to its correct position at the end.
- `input()` and `int()` collect the number of elements and each element from the user.
- `print()` displays the final sorted array.

ERRORS:-

SNO	Errors Observed	Rectification
1	Invalid (non-numeric) input entered by the user.	Enter a valid integer as input.
2	Entering more or fewer elements than the specified count n .	Ensure exactly n elements are entered, one per prompt.
3	Using <code>range(n)</code> instead of <code>range(n - 1 - i)</code> in the inner loop, causing an <code>IndexError</code> .	Use <code>range(n - 1 - i)</code> so <code>arr[j + 1]</code> never goes out of bounds.
4	Incorrect indentation leading to an <code>IndentationError</code> .	Maintain proper Python indentation.
5	Swapping elements incorrectly without simultaneous assignment, losing a value.	Use <code>arr[j], arr[j + 1] = arr[j + 1], arr[j]</code> for a correct swap.

TIME COMPLEXITY:- $O(n^2)$

SPACE COMPLEXITY:- $O(1)$

REASON:- Two nested loops compare and swap every pair of elements in the worst case; sorting is done in place using only a few extra variables.

•

3B)AIM:- Write a Python program to insertion sort the elements, taking input from the user.

PROGRAM:-

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]      j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j + 1] =
arr[j]      j -= 1
        arr[j + 1] = key
    return arr

# Input n = int(input("Enter number of
elements: ")) arr = [] for i in range(n):
    arr.append(int(input(f"Enter element {i+1}: ")))

# Function call sorted_arr =
insertion_sort(arr)

# Output
print("Sorted array:", sorted_arr)
```

OUTPUT:-

```
EEK-3/3-B.py
Enter number of elements: 5
Enter element 1: 2
Enter element 2: 3
Enter element 3: 1
Enter element 4: 5
Enter element 5: 3
Sorted array: [1, 2, 3, 3, 5]
PS C:\Users\Sakate\OneDrive\Desktop\DSA>
```

EXPLANATION:-

- insertion_sort() builds the sorted list one element at a time, similar to sorting playing cards in hand.

- key stores the current element that is being inserted into the sorted portion of the array.
- The while loop shifts elements greater than key one position to the right.
- j decreases until the correct position for key is found or the start of the array is reached.
- `arr[j + 1] = key` places the element in its correct sorted position.
- `input()` and `int()` collect the number of elements and each element from the user.

ERRORS:-

SNO	Errors Observed	Rectification
1	Invalid (non-numeric) input entered by the user.	Enter a valid integer as input.
2	Entering more or fewer elements than the specified count n.	Ensure exactly n elements are entered, one per prompt.
3	Missing the boundary check <code>j >= 0</code> , causing an <code>IndexError</code> .	Always check <code>j >= 0</code> before comparing <code>arr[j]</code> inside the while loop.
4	Incorrect indentation leading to an <code>IndentationError</code> .	Maintain proper Python indentation.
5	Forgetting to decrement j inside the while loop, causing an infinite loop.	Decrease j by 1 in each iteration using <code>j -= 1</code> .

TIME COMPLEXITY:- $O(n^2)$

SPACE COMPLEXITY:- $O(1)$

REASON:- In the worst case (reverse sorted input), every element is compared against and shifted through the entire sorted portion; sorting is done in place.

3C)AIM:- Write a Python program to selection sort the elements, taking input from the user.

PROGRAM:-

```
def selection_sort(arr):  
    n = len(arr)  
    for i in range(n - 1):  
        min_index = i  
        for j in range(i + 1, n):  
            if arr[j] < arr[min_index]:  
                min_index = j  
        arr[i], arr[min_index] = arr[min_index], arr[i]  
    return arr
```

Input

```
n = int(input("Enter number of elements: "))  
arr = []  
for i in range(n):  
    arr.append(int(input(f"Enter element {i+1}: ")))
```

Function call

```
sorted_arr = selection_sort(arr)
```

Output

```
print("Sorted array:", sorted_arr)
```

OUTPUT:-

```
Enter number of elements: 4
Enter element 1: 32
Enter element 2: 23
Enter element 3: 12
Enter element 4: 27
Sorted array: [12, 23, 27, 32]
PS C:\Users\Sakate\OneDrive\Desktop\DSA>
```

EXPLANATION:-

- **selection_sort()** repeatedly selects the minimum element from the unsorted part and places it at the beginning.
- **min_index** tracks the index of the smallest element found during the current pass.
- The inner loop scans the unsorted portion of the array to find the position of the minimum value.
- **arr[i], arr[min_index] = arr[min_index], arr[i]** swaps the found minimum into its correct sorted position.
- **input()** and **int()** collect the number of elements and each element from the user.
- **print()** displays the final sorted array.

ERRORS:-

SNO	Errors Observed	Rectification
1	Invalid (non-numeric) input entered by the user.	Enter a valid integer as input.
2	Entering more or fewer elements than the specified count n.	Ensure exactly n elements are entered, one per prompt.

3	Not resetting min_index to i at the start of each outer pass, giving a wrong sort.	Reset min_index = i at the beginning of every outer loop iteration.
4	Incorrect indentation leading to an IndentationError.	Maintain proper Python indentation.
5	Using <= instead of < when comparing arr[j] and arr[min_index], causing unnecessary swaps.	Use strict < so equal elements are not swapped needlessly.

TIME COMPLEXITY:- $O(n^2)$

SPACE COMPLEXITY:- $O(1)$

REASON:- The inner loop scans all remaining unsorted elements for every outer iteration, giving n^2 comparisons; sorting is done in place.

3D)AIM:- Write a Python program to merge sort the elements, taking input from the user.

PROGRAM:-

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)

def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result

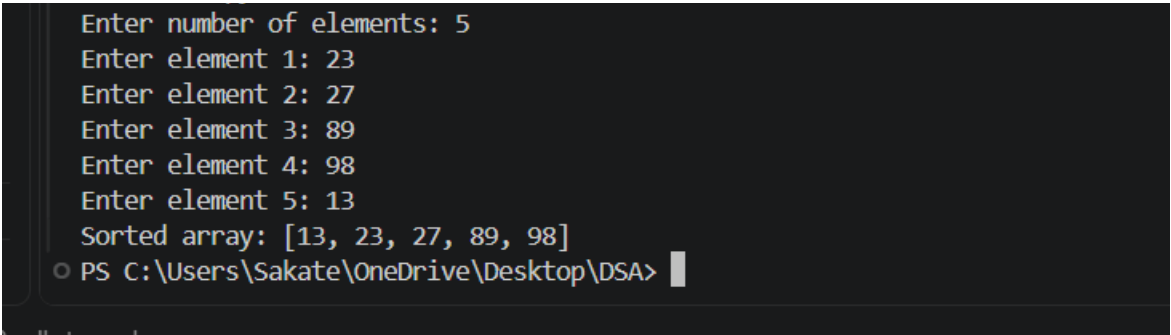
# Input
n = int(input("Enter number of elements: "))
```

```
.  
  
arr = []  
for i in range(n):  
    arr.append(int(input(f'Enter element {i+1}: ')))
```

```
# Function call  
sorted_arr = merge_sort(arr)
```

```
# Output  
print("Sorted array:", sorted_arr)
```

OUTPUT:-



```
Enter number of elements: 5  
Enter element 1: 23  
Enter element 2: 27  
Enter element 3: 89  
Enter element 4: 98  
Enter element 5: 13  
Sorted array: [13, 23, 27, 89, 98]  
PS C:\Users\Sakate\OneDrive\Desktop\DSA>
```

EXPLANATION:-

- **merge_sort()** recursively splits the array into halves until each sub-array has one element.
- **mid = len(arr) // 2** finds the midpoint used to divide the array into left and right halves.
- **left** and **right** hold the recursively sorted halves of the array.
- **merge()** combines two sorted halves into a single sorted list by comparing their front elements.
- **result.extend(...)** appends any elements left over once one half is fully consumed.

- `input()` and `int()` collect the number of elements and each element from the user.

ERRORS:-

SNO	Errors Observed	Rectification
1	Invalid (non-numeric) input entered by the user.	Enter a valid integer as input.
2	Entering more or fewer elements than the specified count n.	Ensure exactly n elements are entered, one per prompt.
3	Missing base case (<code>len(arr) <= 1</code>), causing infinite recursion.	Include the base case to stop recursion when the sub-array has one or zero elements.
4	Incorrect indentation leading to an <code>IndentationError</code>.	Maintain proper Python indentation.
5	Forgetting to extend the remaining elements after the while loop in <code>merge()</code>, losing elements.	Use <code>result.extend(left[i:])</code> and <code>result.extend(right[j:])</code> after the loop.

TIME COMPLEXITY:- $O(n \log n)$

SPACE COMPLEXITY:- $O(n)$

REASON:- The array is divided $\log n$ times and each level requires $O(n)$ work to merge the halves; additional arrays are created at every recursive call.

3E)AIM:- Write a Python program to quick sort the elements, taking input from the user.

PROGRAM:-

```
def quick_sort(arr):  
    if len(arr) <= 1:  
        return arr  
    pivot = arr[len(arr) // 2]  
    left = [x for x in arr if x < pivot]  
    middle = [x for x in arr if x == pivot]  
    right = [x for x in arr if x > pivot]  
    return quick_sort(left) + middle + quick_sort(right)
```

Input

```
n = int(input("Enter number of elements: "))
```

```
arr = []
```

```
for i in range(n):
```

```
    arr.append(int(input(f"Enter element {i+1}: ")))
```

Function call

```
sorted_arr = quick_sort(arr)
```

Output

```
print("Sorted array:", sorted_arr)
```

OUTPUT:-

```
Enter number of elements: 4
Enter element 1: 13
Enter element 2: 23
Enter element 3: 27
Enter element 4: 89
Sorted array: [13, 23, 27, 89]
PS C:\Users\Sakate\OneDrive\Desktop\DSA>
```

EXPLANATION:-

- **quick_sort()** recursively partitions the array around a chosen pivot element.
- **pivot = arr[len(arr) // 2]** selects the middle element of the array as the pivot for partitioning.
- **left, middle, and right** use list comprehensions to group elements less than, equal to, and greater than the pivot.
- The function recursively sorts left and right and concatenates them with middle in between.
- **input()** and **int()** collect the number of elements and each element from the user.
- **print()** displays the final sorted array.

ERRORS:-

SNO	Errors Observed	Rectification
1	Invalid (non-numeric) input entered by the user.	Enter a valid integer as input.
2	Entering more or fewer elements than the specified count n.	Ensure exactly n elements are entered, one per prompt.

3	Missing base case ($\text{len}(\text{arr}) \leq 1$), causing infinite recursion.	Include the base case to stop recursion when the sub-array has one or zero elements.
4	Incorrect indentation leading to an IndentationError.	Maintain proper Python indentation.
5	Always choosing the first element as pivot on already-sorted input, degrading performance to $O(n^2)$.	Choose a better pivot strategy, such as the middle element or a random element.

TIME COMPLEXITY:- $O(n \log n)$ average, $O(n^2)$ worst case

SPACE COMPLEXITY:- $O(\log n)$

REASON:- On average the pivot splits the array into two roughly equal halves, giving $\log n$ levels of recursion with $O(n)$ work per level; the recursion stack depth is $O(\log n)$ on average.

LAB-4

AIM:- Write a Python program to implement a singly linked list that performs the following operations, taking input from the user: create a linked list, insert at the beginning, insert at the end, insert at a specific index, delete by value, delete the first node, delete the last node, count the number of nodes, display/traverse the list, and exit.

PROGRAM:-

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class SinglyLinkedList:
    def __init__(self):
        self.head = None

    def create_list(self):
        self.head = None
        n = int(input("Enter number of nodes to create: "))
        for i in range(n):
            data = int(input(f"Enter value for node {i + 1}: "))
            self.insert_at_end(data)
        print("Linked list created successfully.")

    def insert_at_beginning(self):
        data = int(input("Enter value to insert at beginning: "))
```

```
.  
  
new_node = Node(data)  
new_node.next = self.head  
self.head = new_node  
print(f' {data} inserted at the beginning.")
```

```
def insert_at_end(self, data=None):  
    if data is None:  
        data = int(input("Enter value to insert at end: "))  
    new_node = Node(data)  
    if self.head is None:  
        self.head = new_node  
    else:  
        temp = self.head  
        while temp.next:  
            temp = temp.next  
        temp.next = new_node  
    if data is not None:  
        print(f' {data} inserted at the end.")
```

```
def insert_at_index(self):  
    index = int(input("Enter index to insert at (0-based): "))  
    data = int(input("Enter value to insert: "))  
    if index == 0:  
        new_node = Node(data)  
        new_node.next = self.head  
        self.head = new_node  
        print(f' {data} inserted at index {index}.")  
        return  
    temp = self.head  
    count = 0  
    while temp is not None and count < index - 1:
```

```

    temp = temp.next
    count += 1
if temp is None:
    print("Index out of range.")
    return
new_node = Node(data)
new_node.next = temp.next
temp.next = new_node
print(f'{data} inserted at index {index}.')

def delete_by_value(self):
    value = int(input("Enter value to delete: "))
    if self.head is None:
        print("List is empty.")
        return
    if self.head.data == value:
        self.head = self.head.next
        print(f'Value {value} deleted.')
        return
    prev = self.head
    curr = self.head.next
    while curr:
        if curr.data == value:
            prev.next = curr.next
            print(f'Value {value} deleted.')
            return
        prev = curr
        curr = curr.next
    print(f'Value {value} not found in list.')

def delete_first_node(self):

```

```

.

if self.head is None:
    print("List is empty.")
    return
deleted = self.head.data
self.head = self.head.next
print(f"Deleted first node with value {deleted}.")

def delete_last_node(self):
    if self.head is None:
        print("List is empty.")
        return
    if self.head.next is None:
        deleted = self.head.data
        self.head = None
        print(f"Deleted last node with value {deleted}.")
        return
    temp = self.head
    while temp.next.next:
        temp = temp.next
    deleted = temp.next.data
    temp.next = None
    print(f"Deleted last node with value {deleted}.")

def count_nodes(self):
    count = 0
    temp = self.head
    while temp:
        count += 1
        temp = temp.next
    print(f"Number of nodes: {count}")

```

```
.  
  
def display(self):  
    if self.head is None:  
        print("List is empty.")  
        return  
    temp = self.head  
    elements = []  
    while temp:  
        elements.append(str(temp.data))  
        temp = temp.next  
    print(" -> ".join(elements) + " -> None")
```

```
# Main menu-driven program  
sll = SinglyLinkedList()
```

```
while True:  
    print("\n----- Singly Linked List Menu -----")  
    print("1. Create a linked list")  
    print("2. Insert at beginning")  
    print("3. Insert at end")  
    print("4. Insert at specific index")  
    print("5. Delete by value")  
    print("6. Delete first node")  
    print("7. Delete last node")  
    print("8. Count number of nodes")  
    print("9. Display/Traversal")  
    print("10. Exit")
```

```
choice = int(input("Enter your choice: "))
```

```
if choice == 1:
```

```
.  
    sll.create_list()  
elif choice == 2:  
    sll.insert_at_beginning()  
elif choice == 3:  
    sll.insert_at_end()  
elif choice == 4:  
    sll.insert_at_index()  
elif choice == 5:  
    sll.delete_by_value()  
elif choice == 6:  
    sll.delete_first_node()  
elif choice == 7:  
    sll.delete_last_node()  
elif choice == 8:  
    sll.count_nodes()  
elif choice == 9:  
    sll.display()  
elif choice == 10:  
    print("Exiting program.")  
    break  
else:  
    print("Invalid choice. Try again.")
```

OUTPUT:-

•

```
----- Singly Linked List Menu -----
```

1. Create a linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display/Traversal
10. Exit

Enter your choice: 1

Enter number of nodes to create: 5

Enter value for node 1: 23

23 inserted at the end.

Enter value for node 2: 27

27 inserted at the end.

Enter value for node 3: 32

32 inserted at the end.

Enter value for node 4: 73

73 inserted at the end.

Enter value for node 5: 89

89 inserted at the end.

Linked list created successfully.

```
----- Singly Linked List Menu -----
```

1. Create a linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display/Traversal
10. Exit

Enter your choice: 2

Enter value to insert at beginning: 1

1 inserted at the beginning.

•

```
----- Singly Linked List Menu -----
```

1. Create a linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display/Traversal
10. Exit

Enter your choice: 3

Enter value to insert at end: 2

2 inserted at the end.

```
----- Singly Linked List Menu -----
```

1. Create a linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display/Traversal
10. Exit

Enter your choice: 4

Enter index to insert at (0-based): 3

Enter value to insert: 3

3 inserted at index 3.

```
----- Singly Linked List Menu -----
```

1. Create a linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes

•

```
----- Singly Linked List Menu -----
```

1. Create a linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display/Traversal
10. Exit

Enter your choice: 6

Deleted first node with value 1.

```
----- Singly Linked List Menu -----
```

1. Create a linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display/Traversal
10. Exit

Enter your choice: 7

Deleted last node with value 2.

```
----- Singly Linked List Menu -----
```

1. Create a linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display/Traversal
10. Exit

Enter your choice: 8

```
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display/Traversal
10. Exit
Enter your choice: 8
Number of nodes: 5
```

```
----- Singly Linked List Menu -----
```

```
1. Create a linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display/Traversal
10. Exit
Enter your choice: 9
23 -> 27 -> 32 -> 73 -> 89 -> None
```

```
----- Singly Linked List Menu -----
```

```
1. Create a linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display/Traversal
10. Exit
Enter your choice: 10
Exiting program.
```

```
PS C:\Users\Sakate\OneDrive\Desktop\DSA> █
```

EXPLANATION:-

- The Node class stores a data value and a reference (next) to the following node in the list.
- The SinglyLinkedList class maintains a head pointer and provides methods for each menu operation.
- `create_list()` resets the list and repeatedly calls `insert_at_end()` to build it from user input.
- `insert_at_beginning()` creates a new node, points it to the current head, and makes it the new head.
- `insert_at_end()` traverses to the last node and links a new node after it, or sets it as head if the list is empty.
- `insert_at_index()` walks to the node just before the given position and relinks pointers to insert the new node there.
- `delete_by_value()` searches the list for a matching value and removes that node by updating the previous node's next pointer.
- `delete_first_node()` and `delete_last_node()` move the head pointer or unlink the final node after locating it by traversal.
- `count_nodes()` and `display()` both traverse the list from head to end, counting nodes or collecting their values.
- The main program runs an infinite while loop that prints the menu, reads the user's choice, and calls the matching method until option 10 (Exit) breaks the loop.

ERRORS:-

SNO	Errors Observed	Rectification
1	Entering a non-integer value (e.g. letters) at any input prompt causes a ValueError.	Enter a valid integer whenever the program asks for a value, index, or choice.
2	Choosing an operation such as display, delete, or count before creating the list results in the 'List is empty.' message.	Select option 1 to create the list first before using the other operations.
3	Entering an index beyond the current size in 'Insert at specific index' prints 'Index out of range.'	Check the current node count (option 8) and enter an index within range.
4	Trying to delete a value that is not present in the list prints 'Value not found in list.'	Verify the value exists by displaying the list (option 9) before deleting it.
5	Entering a choice outside 1-10 shows 'Invalid choice. Try again.'	Re-enter a number between 1 and 10 from the menu.

TIME COMPLEXITY:- Insert at beginning and delete first node are $O(1)$; insert at end/index, delete by value/last node, count, and display are $O(n)$

SPACE COMPLEXITY:- $O(n)$ for storing n nodes, $O(1)$ auxiliary space per operation

.

REASON:- Operations that only touch the head (insert at beginning, delete first node) run in constant time because no traversal is needed, while operations that must reach a specific position, value, or the end of the list (insert at end/index, delete by value/last node, count, display) require walking through up to n nodes in the worst case.

Lab-5

1.Aim: Write a python program to implement a stack using an Array. Implement the stack operations such as push, pop, peak and also display the stack elements. PROGRAM: Stack using an Array

Program:

Stack Implementation using Array

```
class Stack:
```

```
    def __init__(self):
```

```
        self.stack = []
```

```
    # Push operation
```

```
    def push(self, data):
```

```
        self.stack.append(data)
```

```
        print(data, "pushed into stack.")
```

```
    # Pop operation
```

```
    def pop(self):
```

```
        if len(self.stack) == 0:
```

```
            print("Stack Underflow.")
```

```
        else:
```

```
            data = self.stack.pop()
```

.

```
print(data, "popped from stack.")
```

```
# Peek operation
```

```
def peek(self):
```

```
    if len(self.stack) == 0:
```

```
        print("Stack is empty.")
```

```
    else:
```

```
        print("Top element:", self.stack[-1])
```

```
# Display stack
```

```
def display(self):
```

```
    if len(self.stack) == 0:
```

```
        print("Stack is empty.")
```

```
    else:
```

```
        print("Stack elements:")
```

```
        for i in range(len(self.stack) - 1, -1, -1):
```

```
            print(self.stack[i])
```

```
# Main Program
```

```
s = Stack()
```

.

while True:

```
    print("\n===== STACK USING ARRAY =====")
```

```
    print("1. Push")
```

```
    print("2. Pop")
```

```
    print("3. Peek")
```

```
    print("4. Display")
```

```
    print("5. Exit")
```

```
    print("=====")
```

```
    choice = int(input("Enter your choice: "))
```

```
    if choice == 1:
```

```
        data = int(input("Enter data: "))
```

```
        s.push(data)
```

```
    elif choice == 2:
```

```
        s.pop()
```

```
    elif choice == 3:
```

```
        s.peek()
```

.

```
elif choice == 4:
```

```
    s.display()
```

```
elif choice == 5:
```

```
    print("Program terminated.")
```

```
    break
```

```
else:
```

```
    print("Invalid choice.")
```

OUTPUT :

•

```
===== WEEK-5 =====
```

1. Stack using Array
2. Stack using Linked List
3. Infix to Postfix
4. Exit

Enter choice: 1

```
--- Array Stack ---
```

1. Push
2. Pop
3. Display
4. Exit

Enter choice: 1

Enter element: 23

Pushed: 23

```
--- Array Stack ---
```

1. Push
2. Pop
3. Display
4. Exit

Enter choice: 1

Enter element: 27

Pushed: 27

```
--- Array Stack ---
```

1. Push
2. Pop
3. Display
4. Exit

Enter choice: 1

Enter element: 89

Pushed: 89

```
--- Array Stack ---
```

1. Push
2. Pop
3. Display
4. Exit

```

PS C:\Users\Sakate\OneDrive\Desktop\DSA>
WEEK 5/stack.py"
--- Array Stack ---
1. Push
2. Pop
3. Display
4. Exit
Enter choice: 1
Enter element: 89
Pushed: 89

--- Array Stack ---
1. Push
2. Pop
3. Display
4. Exit
Enter choice: 2
Popped: 89

--- Array Stack ---
1. Push
2. Pop
3. Display
4. Exit
Enter choice: 3
Stack: ['23', '27']

--- Array Stack ---
1. Push
2. Pop
3. Display
4. Exit
Enter choice: 4

===== WEEK-5 =====
1. Stack using Array
2. Stack using Linked List
3. Infix to Postfix
4. Exit
Enter choice: █

```

Explanation:

1. `stack = []` creates an empty list which is used as the array of the stack.
2. `size` stores the maximum number of elements the stack can hold, entered by the user.

- 3.The push(item) function first checks whether len(stack) == size. This is the overflow condition.
- 4.If the stack is full, the program prints "Stack Overflow" and the element is not inserted.
- 5.Otherwise, stack.append(item) inserts the element at the top of the stack.
- 6.The user enters the number of elements to push, and a for loop reads each element and calls push().
- 7.Finally, the stack is displayed from top to bottom using stack[::-1].

Error table:

Si.no	Error obtained	Rectification
1.	if len(stack) = size:	if len(stack) == size:

2.Aim: Write a python program to implement a stack using an Linked list. Implement the stack operations such as push, pop, peak and also display the stack elements.

Program: Stack using an Linked List.

Stack Implementation using Linked List

```
class Node:
```

```
    def __init__(self, data):
```

```
        self.data = data
```

```
        self.next = None
```

```
class Stack:
```

```
    def __init__(self):
```

```
        self.top = None
```

```
# Push operation
```

```
def push(self, data):
```

```
    new_node = Node(data)
```

```
    new_node.next = self.top
```

```
    self.top = new_node
```

```
    print(data, "pushed into stack.")
```

.

Pop operation

```
def pop(self):
```

```
    if self.top is None:
```

```
        print("Stack Underflow.")
```

```
    else:
```

```
        data = self.top.data
```

```
        self.top = self.top.next
```

```
        print(data, "popped from stack.")
```

Peek operation

```
def peek(self):
```

```
    if self.top is None:
```

```
        print("Stack is empty.")
```

```
    else:
```

```
        print("Top element:", self.top.data)
```

Display stack

```
def display(self):
```

```
    if self.top is None:
```

```
        print("Stack is empty.")
```

.

else:

temp = self.top

print("Stack elements:")

while temp is not None:

print(temp.data)

temp = temp.next

Main Program

s = Stack()

while True:

print("\n===== STACK USING LINKED LIST =====")

print("1. Push")

print("2. Pop")

print("3. Peek")

print("4. Display")

print("5. Exit")

print("=====")

```
.
```

```
choice = int(input("Enter your choice: "))
```

```
if choice == 1:
```

```
    data = int(input("Enter data: "))
```

```
    s.push(data)
```

```
elif choice == 2:
```

```
    s.pop()
```

```
elif choice == 3:
```

```
    s.peek()
```

```
elif choice == 4:
```

```
    s.display()
```

```
elif choice == 5:
```

```
    print("Program terminated.")
```

```
    break
```

```
else:
```

```
    print("Invalid choice.")
```

OUTPUT:

```
===== WEEK-5 =====
1. Stack using Array
2. Stack using Linked List
3. Infix to Postfix
4. Exit
Enter choice: 2

--- Linked List Stack ---
1. Push
2. Pop
3. Display
4. Exit
Enter choice: 1
Enter element: 23
Pushed: 23

--- Linked List Stack ---
1. Push
2. Pop
3. Display
4. Exit
Enter choice: 1
Enter element: 27
Pushed: 27

--- Linked List Stack ---
1. Push
2. Pop
3. Display
4. Exit
Enter choice: 1
Enter element: 89
Pushed: 89

--- Linked List Stack ---
1. Push
2. Pop
```

```

1. Push
2. Pop
3. Display
4. Exit
Enter choice: 1
Enter element: 89
Pushed: 89

--- Linked List Stack ---
1. Push
2. Pop
3. Display
4. Exit
Enter choice: 2
Popped: 89

--- Linked List Stack ---
1. Push
2. Pop
3. Display
4. Exit
Enter choice: 3
Stack: 27 23

--- Linked List Stack ---
1. Push
2. Pop
3. Display
4. Exit
Enter choice: 4

===== WEEK-5 =====
1. Stack using Array
2. Stack using Linked List
3. Infix to Postfix
4. Exit
Enter choice: █

```

EXPLANATION:

Node(data) : Creates a node and stores the element in data. ·

next = None : Initially, the node has no link. ·

top = None : Initializes an empty stack.

·

Push:

- `new_node = Node(data)` : Creates a new node. ·
- `new_node.next = self.top` : Links it to the current top. ·
- `self.top = new_node` : Makes it the new top.
- Process: Create : Link : Update top.

Pop: ·

- Checks `self.top` is None for underflow. ·
- `popped = self.top.data` : Gets the top value. ·
- `self.top = self.top.next` : Moves top to the next node. ·
- Process: Get value : Move top : Remove old top.

Peek:

- `self.top.data` : Displays the top value without removing it.

Display:

- `temp = self.top` : Starts from top. ·
- `while temp` : Traverses the nodes.
- `temp = temp.next` : Moves to the next node.

Main Program:

- SLL = Stack LinkedList() : Creates the stack. ·
- choice : Selects Push, Pop, Peek, Display, or Exit. ·
- while True : Repeats the menu. ·
- break : Exits the program. ·

LIFO: Last inserted : First removed.

ERROR TABLE:

s.no	Error Obtained	Rectification
1.	Attribute Error may occur in Linked List Stack if self.top.data is accessed when top is None.	Check if self.top is None before performing pop() or peek().